

Vehicle Management System

(Problem Description)

Background & Motivation

Modern transport systems involve managing multiple types of **vehicles**—cars, buses, bikes, trucks, and many others. Even in a simplified environment, **keeping track of vehicle records** is challenging without a structured system. Information such as **model name, registration number, brand, engine capacity, or manufacturing year** needs to be **stored, retrieved, updated, and removed** as the fleet grows or changes. This project requires you to develop a basic **command-line software** that handles these tasks. The primary objective is to design a program capable of performing structured **operations on multiple vehicle records** and simulating a lightweight **information-management** process.

Problem Statement

Design and implement a **menu-driven Vehicle Management System** that allows the user to store and manage information about different types of vehicles. Each vehicle must contain a minimum **set of predefined attributes**. The program must allow users to **add new vehicles, list all vehicles, search for specific vehicles, update existing information, and delete records**. The system should handle multiple vehicles and continue to function correctly regardless of how many operations the user performs.

Your program will simulate how vehicle operations might work in a very simplified database, using only **basic data structures and manual handling of records**.

Core Requirements

1. Vehicle Data Structure

Each vehicle must store at least the following details:

- Vehicle ID (unique numeric identifier)
- Vehicle Type (car, bus, bike, truck, etc.)
- Model Name
- Brand/Manufacturer
- Year of Manufacture
- Engine Capacity (cc or descriptive text)

You may include optional attributes such as color, seat count, or fuel type.

2. Required Operations

Your program must provide a main menu with the following functions:

A. Add New Vehicle

- Prompt the user to enter all attributes for a new vehicle.
- Assign or validate that the Vehicle ID is unique.
- Store the vehicle in an appropriate internal structure (array or similar).
- Confirm successful insertion.

B. Show All Vehicles

- Display every stored vehicle in a clean, readable list format.
- Show all attributes for each vehicle.
- If no vehicles are available, show a suitable message.

C. Search for a Vehicle

Provide at least **two types of search**:

1. **Search by Vehicle ID**
2. **Search by Vehicle Type or Model Name**

The program should:

- Prompt for search input
- Scan through all stored vehicles
- Display matching vehicle(s) with full details
- Show “not found” if no match exists

D. Update Vehicle Information

- Allow the user to specify a Vehicle ID to update.
- If found, show current details.
- Let the user modify one or more fields.
- Save the updated record.

E. Delete a Vehicle

- Ask the user for a Vehicle ID.
- Locate and remove it from storage.
- Shift elements if necessary to maintain continuity.
- Confirm deletion.

F. Exit

- Terminate the program cleanly.

3. Functional Constraints

1. **Manual Data Storage Only**
 - All vehicles must be stored in arrays, structures, or basic data constructs.
 - No external database, files, or advanced libraries.
2. **Maximum Record Limit**
 - You may set a maximum number of vehicles (e.g., 100 or 200) to simplify memory.

3. **Unique Vehicle IDs**
 - System must ensure IDs are not duplicated.
 - If user enters an existing ID, they must be prompted to try again.
4. **Input Validation**
 - System should handle invalid user inputs (e.g., letters instead of numbers).
5. **Consistent Menu Loop**
 - The program must not terminate until the user explicitly selects Exit.
6. **Full Record Consistency**
 - Updating or deleting must not corrupt other records.
 - All operations should maintain the integrity of stored data.
7. **Separation of Logic (Optional but Encouraged)**
 - You may modularize your program using functions for:
 - addVehicle()
 - showVehicles()
 - searchVehicle()
 - updateVehicle()
 - deleteVehicle()

4. Expected Learning Outcomes

By completing this project, you will:

- Gain hands-on experience managing structured data manually.
- Understand the limitations of flat arrays and manual record handling.
- Practice designing and implementing multi-step operations.
- Improve problem-solving and modular programming skills.
- Learn how real-world systems require extensibility, maintainability, and scalable structures.

Features and Operations

1. Filter Vehicles Based on Multiple Conditions

Filter vehicles to:

- Show all vehicles manufactured after 2015 AND engine capacity above 1500cc
- Show all bikes with engine capacity below 150cc
- Show all buses of brand “Volvo” between years 2010–2020

2. Sort Vehicles by Multiple Criteria

Examples:

- Sort vehicles by Year (descending)
- Sort by Brand alphabetically

- Sort by Engine Capacity, but if equal, then sort by Model Name

3. Duplicate Detection Across Attributes

Detect:

- Two vehicles with the same model name but different IDs
- Two vehicles with the same brand and same year
- Any duplicate Model Name across the whole list

4. Bulk Delete Operation

Examples:

- Delete all vehicles older than year 2010
- Delete all bikes with engine capacity < 100cc
- Delete all vehicles of brand “Toyota”

5. Partial Update of All Matching Records

Examples:

- Increase engine capacity of all trucks by +200
- Update the brand name “Honda” to “Honda Motors” everywhere
- Change vehicle type “bike” to “motorcycle” for all matching entries

6. Generate Summary Statistics

Compute:

- Count vehicles by type: Car=5, Bus=2, Bike=12
- Highest engine capacity vehicle
- Average year of manufacture for all cars
- Percentage of vehicles made after 2020

7. Group Vehicles by Category

Example output:

```
Cars:
  [list]
Bikes:
  [list]
Trucks:
  [list]
Others:
  [list]
```

8. Dynamic Data Expansion

- When maximum size is reached, allow adding more vehicles
- “Increase capacity” by 20 more records

9. Clone a Vehicle Record to Create a Template

Example:

- Copy a vehicle with ID X to create a new one with a different ID
- Used for creating “similar” vehicles quickly

10. Create a Simple Report View

Example:

Show a report:

```
Total Vehicles: 20
Average Engine Capacity: 1250cc
Oldest Vehicle Year: 1998
Newest Vehicle Year: 2024
Brand Distribution:
  Toyota: 5
  Honda: 3
  Bajaj: 4
  Others: 8
```

11. Manage Multiple Types of Vehicles with Different Attributes

Introduce different attributes:

- Car: seatCount
- Bike: gearCount
- Bus: routeNumber

12. Convert Records into a Single Searchable String

User can search by any text in the record:

Input: “2020”, Output: show all vehicles containing “2020” in any field
Input: “Honda”, Output: all Honda vehicles

13. Undo Last Operation

Let the user undo the last:

- Add
- Update
- Delete